实验 6　进程互斥与死锁避免

一、实验目标

(1)分析进程竞争资源的现象

(2)学习解决进程互斥的方法

(3)基于 openEuler 系统实现银行家算法和随机算法的资源分配

二、背景知识

2.1 进程互斥

所谓进程互斥，指的是对某个系统资源，一个进程正在使用它，另外一个想用它的进程就必须等待，而不能同时使用。

进程互斥是多道程序系统中进程间存在的一种源于资源共享的制约关系，也称间接制约关系，主要是由被共享资源的使用性质所决定的。

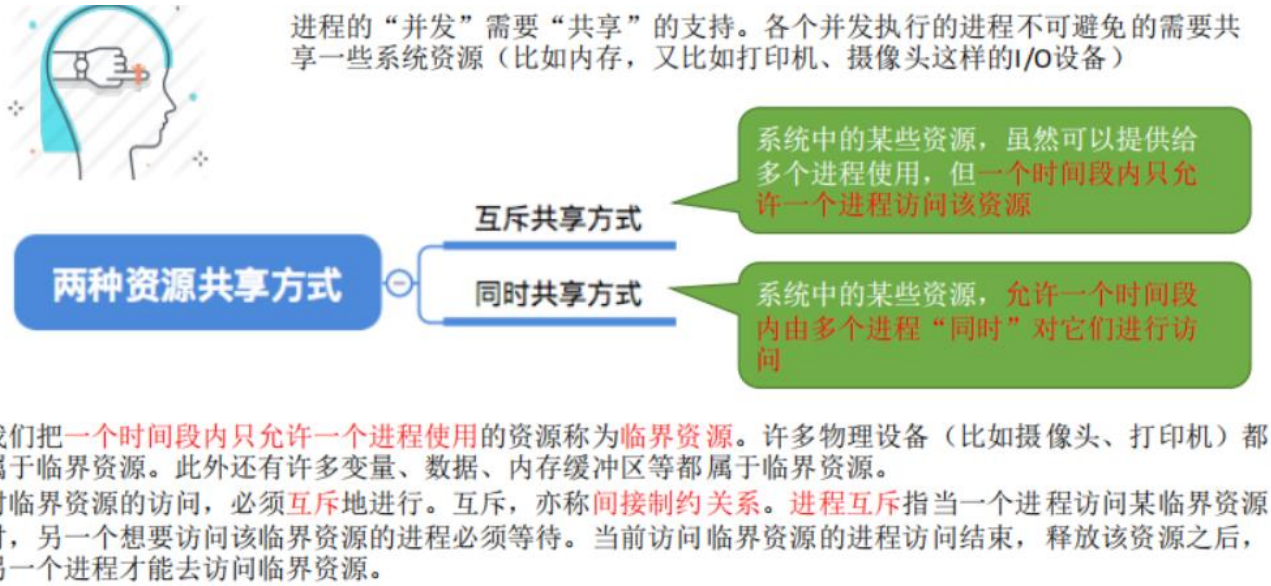


图 6.1　进程互斥

进程互斥遵循的原则：

(1)空则让进：当无进程在互斥区时，任何有权使用互斥区的进程可进入。

(2)忙则等待：不允许两个以上的进程同时进入互斥区，当已有进程进入临界区时，其他试图进入临界区的进程必须等待。

(3)等则有限：对请求访问的进程，应该保证能在有限时间内进入临界区（保证不会饥饿）。

(4)等则让权：当进程不能进入临界区时，应立即释放处理机，避免进程忙等待。

2.2 死锁避免

所谓死锁，指的是两个或两个以上的进程在执行过程中，由于竞争资源或者由于彼此通信而造成的一种阻塞的现象，如果无外力作用，那么它们都将无法推进下去。此时，称系统处于死锁状态或系统产生了死锁，这些永远在互相等待的进程称为死锁进程。

死锁的规范定义：集合中的每一个进程都在等待只能由本集合中的其他进程才能引发的事

件，那么该组进程是死锁的。

产生条件：

(1)互斥（资源独占）： 一个资源每次只能被一个进程使用；

(2)请求与保持（部分分配，占有申请）：一个进程在申请新的资源的同时保持对原有资源的占有（只有这样才是动态申请，动态分配）；

(3)不可剥夺（不可强占）：资源申请者不能强行地从资源占有者手中夺取资源，资源只能由占有者自愿释放；

(4)循环等待：若干进程之间形成一种头尾相连的循环等待资源关系。

避免死锁的方法：

(1)一次封锁法：每个进程（事务）将所有要使用的数据全部加锁，否则，就不能继续执行；

(2)顺序封锁法：预先对数据对象规定一个封锁顺序，所有进程（事务）都按这个顺序加锁；

(3)银行家算法：银行家算法（Banker's Algorithm）是一个避免死锁（Deadlock）的著名算法是由艾兹格·迪杰斯特拉在 1965 年为 T.H.E 系统设计的一种避免死锁产生的算法。它以银行借贷系统的分配策略为基础，判断并保证系统的安全运行。

银行家算法的分配原则是：

①验证进程当前申请数是否合法，若当前申请数与已占资源数的和超过进程的资源需求总量，则当前申请数不合法，需重新输入申请数。

②系统当前剩余资源量不能满足进程申请数，进程置为等待态。

③系统当前剩余资源量若能满足进程申请数，则假定将所申请资源数分配给该进程，检查系统是否处于安全状态，即所有进程是否都能得到执行完标志。

④若系统不处于安全状态，分配无效，置申请进程为等待态。否则分配有效，将当前申请数加入该进程的已占资源数。

⑤检查该进程的已占资源数是否等于进程的资源需求总量。若成立，表示该进程可完成，置进程为完成态，释放所有已占有资源。同时检查是否有等待进程，现在的剩余资源数是否能分配给它们。

⑥重复上述过程，直到所有进程都成功完成。

(4)随机算法：随机算法采用了一定程度的随机性作为其逻辑的一部分。该算法通常使用均匀随机位作为辅助输入来指导自己的行为，超过随机位的所有可能的选择实现了“平均情况下的”良好成绩。从形式上看，该算法的性能将会是一个随机变量，由随机位决定；因此无论是运行时间，或输出，或两者都是随机变量。

随机算法的分配原则是：

①验证进程当前申请数是否合法，若当前申请数与已占资源数的和超过进程的资源需求总量，则当前申请数不合法，需重新输入申请数。

②系统当前剩余资源量不能满足进程申请数，进程置为等待态。
系统当前剩余资源量若能满足进程申请数，则实施分配。

③检查该进程的已占资源数是否等于进程的资源需求总量。若成立，表示该进程可完成，置进程为完成态，释放所有已占有资源。同时检查是否有等待进程，现在的剩余资源数是否能分配给它们。

④重复上述过程，直到所有进程都成功完成或若干进程形成死锁，即都处于等待状态。

三、实验步骤

3.1 案例 1

使用 lockf 函数来为进程进行加锁以实现进程之间的互斥。

(1)输入命令 touch lock.c 创建.c 文件，输入命令 vi lock.c 编辑.c 文件。

```
[root@localhost ~]# touch lock.c
[root@localhost ~]# vi lock.c
```

(2)按下键盘 i 键进入 Insert 模式，并输入以下代码：

```
#include <stdio.h>
#include <unistd.h>
int main(      )
{
    int p1,p2,i;
    while((p1=fork( ))== -1);  /*创建子进程  p1*/

    if (p1==0)
{
    lockf(1,1,0); /*加锁，第一个参数为  stdout（标准输出设备的描述符）*/
    for(i=0;i<10;i++) printf("daughter %d\n",i);
    lockf(1,0,0); /*解锁*/

}
    else
{
    while((p2=fork( ))==-1);
    /*创建子进程  p2*/
    if (p2==0)
{
    lockf(1,1,0);
    /*加锁*/
    for(i=0;i<10;i++) printf("son %d\n",i);
    lockf(1,0,0); /*解锁*/

}

else
{
    lockf(1,1,0);
    /*加锁*/
    for(i=0;i<10;i++) printf(" parent %d\n",i);
    lockf(1,0,0); /*解锁*/
}

}
}
```

(3)按下键盘 esc 键并输入”:wq”退出编辑模式。

(4)输入 gcc 命令执行.c 文件：

```
[root@localhost ~]# gcc lock.c -o lock
[root@localhost ~]# ./lock
```

(5)实验结果如下所示：

```
daughter 0
daughter 1
daughter 2
daughter 3
daughter 4
daughter 5
daughter 6
daughter 7
daughter 8
daughter 9
 parent 0
 parent 1
 parent 2
 parent 3
 parent 4
 parent 5
 parent 6
 parent 7
 parent 8
 parent 9
son 0
son 1
son 2
son 3
son 4
son 5
son 6
son 7
son 8
son 9
```

3.2 案例 2

在案例 1 的基础上使用 create 函数建立共享文件 to_be_locked.txt，以此文件作为共享资源为进程进行加锁以实现进程之间的互斥。

(1)输入命令 touch lock2.c 创建.c 文件，输入命令 vi lock2.c 编辑.c 文件。

```
[root@localhost ~]# touch lock2.c
[root@localhost ~]# vi lock2.c
```

(2)按下键盘 i 键进入 Insert 模式，并输入以下代码：

```
#include <stdio.h>
```

```
#include <stdlib.h>
#include <unistd.h>
#include <sys/file.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
int main()
{
    int p1,p2,i;
    char outstr[20]; int fd;
    fd = creat("to_be_locked.txt",0777);    /* 调用系统服务 creat，建立共享文件 to_be_locked.txt，返回文件描述符 fd */
    if (fd==0)/* 建立失败，则终止退出 */
    {
    printf("Fail to create file!");
    exit(-1);
}
while((p1=fork())==-1);
    /* 创建子进程 p1 */
if (p1==0)
    {
lockf(fd,1,0);
    /* 加锁 */
       for(i=0; i<5; i++) {
          sprintf(outstr,"daughter %d\n",i); /* 形成输出串 */
          write(fd,outstr,11); /* 写入共享文件 */
}
    lockf(fd,0,0);  /* 解锁 */

}
else
    {
        while((p2=fork())==-1);
        if (p2==0)
            {
            lockf(fd,1,0);
            /*加锁*/
            for(i=0; i<5; i++) { sprintf(outstr,"son %d\n",i);
            write(fd,outstr,6);
            }

            lockf(fd,0,0);  /*解锁*/

            }
```

```
else
        {
            lockf(fd,1,0);
            /*加锁*/
            for(i=0; i<5; i++) { sprintf(outstr,"parent %d\n",i);
            write(fd,outstr,9);
        }

        lockf(fd,0,0);  /*解锁*/

    }

}
close(fd);
 }
```

(3)按下键盘 esc 键并输入":wq"退出编辑模式。

(4)输入 gcc 命令执行.c 文件：

```
[root@localhost ~]# gcc lock.c -o lock
[root@localhost ~]# ./lock
```

(5)输入命令 cat to_be_locked.txt 查看输出结果：

```
[root@localhost ~]# cat to_be_locked.txt
```

(6)to_be_locked.txt 内容如下所示：

```
parent 0
parent 1
parent 2
parent 3
parent 4
son 0
son 1
son 2
son 3
son 4
daughter 0
daughter 1
daughter 2
daughter 3
daughter 4
```

3.3 案例 3

使用银行家算法和随机算法实现资源分配

(1)输入命令 yum install gcc g++安装 c++编译器：

```
[root@localhost ~]# yum install gcc g++
```

(2)输入命令 touch bank.c 创建.c 文件，输入命令 vi bank.c 编辑.c 文件。

```
[root@localhost ~]# touch bank.c
[root@localhost ~]# vi bank.c
```

(3)按下键盘 i 键进入 Insert 模式，并输入以下代码：

```
#include <iostream>
#include <vector>
// #include <fstream>
using namespace std;
const int TASK_RUNNING = 0;
const int TASK_SUCCEED = 1;
const int TASK_WAITTING = 2;
const int RLength = 10;
int Rcs_left = RLength;
class pcb{
public: int p_pid;
      int p_stat;
      int p_apply;
      int p_occupy;
      bool p_issuc;
      int p_require;
      pcb(int id, int require)
      { p_pid = id;
      p_require = require;
      p_stat = TASK_RUNNING;
      p_occupy = 0;
      p_issuc = false;
      p_apply = 0;
}
      friend ostream & operator << (ostream &cout,const pcb & p)
      {
      cout<<p.p_pid<<'\t'<<p.p_stat<<'\t'<<p.p_require<<'\t'<<p.p_occupy<<endl;
      return cout;
}
};
void rand(/*vector<int>&resource,*/vector<pcb>&pgrp);
void banker(/*vector<int>&resource,*/vector<pcb>&pgrp);
int main()
{
vector<pcb>pgrp;
      cout<<"ENTER THE MAX NUMBER FOR THE REQESTED RESOURCE:"<<endl;
      cout<<"ID\tREQUESTED"<<endl;
      int qty;
      for(int i(1);i<=4;i++) {
```

```
do { cout<<i<<'\t';
	cin>>qty;
} while(qty>Rcs_left || qty<1);
	pgrp.insert(pgrp.begin(),pcb(i,qty));
	}
	cout<<"ALOGRITHM"<<endl
	<<"Random(R)"<<'\t'<<"Banker(B)"<<endl
	<<"ANY OTHER KEY TO QUIT"<<endl;
	char choice;
	cin>>choice;
	if(choice == 'R' || choice == 'r')
	rand(/*resource,*/pgrp);
	else if(choice == 'B' || choice == 'b')
	banker(/*resource,*/pgrp);
	else  return 0;
	return 1;
}
void rand(/*vector<int>&resource,*/vector<pcb>&pgrp) {
	vector<pcb>::iterator p,q;
	vector<pcb>::iterator current; int temp;
	cout<<"NOW--------RANDOM ALOGRITHM"<<endl;
	for(;;)
	{
	for(p=pgrp.begin();p!=pgrp.end();p++)
	{
		if(p->p_stat == TASK_RUNNING) {
		current = p;
		break;
}
}
if(current->p_apply == 0)
{
	cout<<"ENTER THE APPLY FOR THE PROCESS\n"<<current->p_pid<<'\t';
	cin>>temp;
	while(temp>p->p_require - p->p_occupy)
	{
	cout<<"beyond the real need!"<<endl;
	cout<<"ENTER THE APPLY FOR THE PROCESS\n"<<current->p_pid<<'\t'; cin>>temp;
	}
	p->p_apply = temp;
}
if(current->p_apply > Rcs_left)
{
for(p=pgrp.begin();p!=pgrp.end();p++)
```

```
{
    if(p->p_stat==TASK_WAITTING && Rcs_left>=p->p_apply)
    break;
}
    if(p!=pgrp.end())
{
    p->p_stat   =   TASK_RUNNING;
    pcb proc(*p);
    pgrp.erase(p);
    pgrp.insert(pgrp.end(),proc);
    continue;
}
}
}
void banker(/*vector<int>&resource,*/vector<pcb>&pgrp)
{
    vector<pcb>::iterator p;
    vector<pcb>::iterator current,q;
    pcb proc(0,0);
    int length;
    cout<<"NOW--------BANKER ALOGRITHM"<<endl;
    for(;;)
    {
    for(p=pgrp.begin();p!=pgrp.end();p++)
    /*  找到运行进程  */
{
    if(p->p_stat == TASK_RUNNING) {
    current = p;
    break;
}
}
    if(current->p_apply == 0)
{
    cout<<"ENTER THE APPLY FOR THE PROCESS\n"<<current->p_pid<<'\t';
    cin>>current->p_apply;
    while(current->p_apply > current->p_require - current->p_occupy) {
    cout<<"beyond the real need!"<<endl;
    cout<<"ENTER THE APPLY FOR THE PROCESS\n"<<current->p_pid<<'\t';
    cin>>current->p_apply;
}
}
if(current->p_apply > Rcs_left)
{
    current->p_stat = TASK_WAITTING;
```

```
        proc = *current; pgrp.erase(current);
        pgrp.insert(pgrp.end(),proc);
        cout<<endl<<p->p_pid<<" is waitting!\n";
        continue;
}
        pcb backup(*current);
        length = Rcs_left;
        current->p_occupy += current->p_apply;
        length -= current->p_apply;
        if(current->p_occupy == current->p_require) {
        length += current->p_require;
        current->p_issuc = true;
}
        int flag = 1;
        while(flag==1)
{
        flag = 0;
        for(p=pgrp.begin();p!=pgrp.end();p++)
{
        if(p->p_stat == TASK_SUCCEED) continue;
        if(p->p_issuc==true) continue;
        if((p->p_require - p->p_occupy) > length) continue;
        else
{
        p->p_issuc = true; length += p->p_occupy;
        flag = 1;
        cout<<p->p_pid<<"      "<<p->p_occupy<<"      "<<length<<endl;
        continue;
}
}
}
for(p=pgrp.begin();p!=pgrp.end();p++)
{
        if(p->p_issuc==false && p->p_stat != TASK_SUCCEED)
        break;
}
if(p!=pgrp.end())
{
        current->p_occupy = backup.p_occupy;
current->p_stat    = TASK_WAITTING;
        cout<<endl<<current->p_pid<<"      is waitting."<<endl;
        proc = *current;
        pgrp.erase(current);
        pgrp.insert(pgrp.end(),proc);
```

```
        for(p=pgrp.begin();p!=pgrp.end();p++)
        p->p_issuc = false;
        continue;
}
Rcs_left-=current->p_apply;
        cout<<endl<<current->p_pid<<" get "<<current->p_apply<<" resource(s)!"<<endl;
        current->p_apply = 0;
        for(p=pgrp.begin();p!=pgrp.end();p++)
        p->p_issuc = false; if(current->p_occupy < current->p_require) {
        proc = *current;
        pgrp.erase(current);
        pgrp.insert(pgrp.end(),proc);
        continue;
}
current->p_stat   =   TASK_SUCCEED;
        current->p_occupy = 0;
        cout<<endl<<current->p_pid<<"     has finished!!!"<<endl;
        Rcs_left+=current->p_require;
        for(p=pgrp.begin();p!=pgrp.end();p++)
{
if(p->p_stat   ==   TASK_WAITTING)
        break;
}
        if(p==pgrp.end())
{
        for(q=pgrp.begin();q!=pgrp.end();q++)
        if(q->p_stat == TASK_RUNNING) break;
        if(q==pgrp.end())
{
        cout<<endl<<"SUCCEED!!"<<endl;
        exit(0);
}
else continue;
}
        p->p_stat   =   TASK_RUNNING;
        continue;
}}
```

(4)按下键盘 esc 键并输入":wq"退出编辑模式。

(5)输入命令 g++ bank.c -o bank 编译.c 文件,输入命令./bank 执行：

```
[root@localhost ~]# g++ bank.c -o bank
[root@localhost ~]#./bank
```

(6)依次输入每个进程所需资源数，每次输入结束按下 Enter 键。

```
ENTER THE MAX NUMBER FOR THE REQESTED RESOURCE:
```

```
ID     REQUESTED
1      2
2      3
3      4
4      5
```

(7)选择执行的算法(R 为随机算法，B 为银行家算法)，此处以银行家算法为例：

```
ALOGRITHM
Random(R)      Banker(B)
ANY  OTHER  KEY  TO  QUIT
B
```

(8)输入可使用的资源数目(多次输入)，出现”**SUCCEED！**”即为成功。

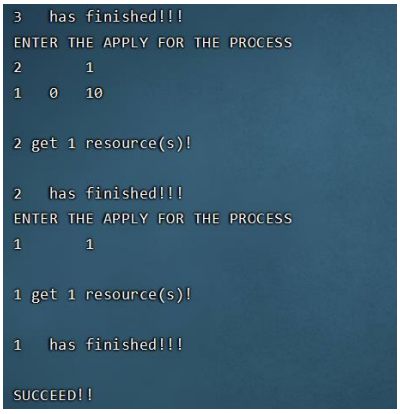


图 6.2　SUCCEED！

四、　实验结果讨论

(1)你认为在实现进程互斥时，使用 lockf 函数或共享文件的方式哪种更可靠？为什么

(2)随机算法与银行家算法有何不同？在什么情况下使用随机算法可能更好？

(3)在实现进程互斥时，除了 lockf 函数和共享文件之外，还有哪些方法可以实现进程之间的互斥？这些方法各有什么优缺点？